



https://github.com/SETAPESU26/24_treasure_hunt

see this is my lab 4 se i want you to help me to complete this lab
explain me question and the procedure to complete this ;ab

Yes. I checked **both your Lab 4 handout and the actual 24_treasure_hunt repository**, so we can follow exactly what your faculty expects.

The lab is basically: **take the existing Treasure Hunt game → understand it → use an LLM/Vibe Coding to fix/add features one by one → test everything → make separate commits → submit evidence.**

Your handout explicitly says the assignment is individual, you must clone/fork the assigned repo, run it, record the original game, use prompts to fix/add features, record the final game, and push your work.



Lab_4_VibeCoding_Student_handout



Lab_4_VibeCoding_Student_handout

1. What exactly is your Lab 4?

Your repository is:

[24_treasure_hunt GitHub repository](#) ↗

It is a **Python/Pygame Treasure Hunt game**.

The original game has:

- Procedurally generated dungeon
- Player movement
- Walls/floor
- Key
- Chest
- Collision
- Restart

The README says the game is run with:

`</> Bash`

```
pip install pygame-ce
python game.py
```

and the controls are:

 Plain text

```
W/A/S/D or Arrow Keys → Move
R → Restart
``` :chatgpt-content-reference{index="3"}

2. The MOST IMPORTANT part – what do you have to add?

Your README has 4 tasks.

You need to complete them one by one.

Task 1 – Floor Traps

You need to add traps to the dungeon.

When the player walks onto a trap:

```text
Player touches trap
```

The README specifically says the player must visibly reset to the start room and show a clear status message.  [GitHub](#)

Task 2 — Enemy Guard

Add an enemy/guard near the chest.

The guard should:

 Plain text

```
Point A → Point B
  ↑       ↓
  ←————→
```

It continuously patrols back and forth.

If:

</> Plain text

Player touches Guard

then:

</> Plain text

Player → Starting position

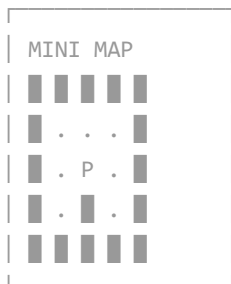
The README requires the guard to move between two patrol points and reset the player when they collide. [GitHub](#)

Task 3 — Mini-map

Add a small map in a corner.

Something like:

</> Plain text



It needs to show:

- Walls
- Floor
- Player position
- Real-time updates

The README specifically requires the mini-map to distinguish walls/floor and mark the player's current position.  [GitHub](#)

Task 4 — Inventory UI

Add an inventory slot/HUD.

Initially:

 Plain text

Inventory



After collecting the key:

 Plain text

Inventory



The slot must immediately change when the key is collected.  [GitHub](#)

3. There is also a testing requirement

This part is VERY important because you don't just need to write the code.

You have to test:

Test 1

Collect key → go to chest.

Test 2

Trigger every trap type.

Test 3

Confirm trap resets player.

Test 4

Touch guard.

Test 5

Confirm guard collision resets player.

Test 6

Check mini-map matches the actual dungeon.

Test 7

Collect key → check inventory changes.

Test 8

Press **R** in the middle of the game and verify restart works.

These are explicitly listed in the repository README.  [GitHub](#)

4. Your complete Lab 4 workflow

This is the workflow I recommend you follow:

 Plain text

STEP 1

Fork/clone assigned repository



STEP 2

Run original game



STEP 3

Understand README + game.py



STEP 4

Record 10-second BEFORE video



STEP 5

Task 1 → Floor Traps



Test Task 1



Git commit Task 1



STEP 6

5. Before you change anything

Do not immediately ask ChatGPT to rewrite `game.py` .

The README specifically says you should inspect the existing code first and understand the flow through:

⟨/⟩ Plain text

GameEngine



Player



Grid / dungeon



Player state



Collision / items

It also explicitly says not to replace the entire project with an unrelated implementation.  GitHub

That's actually important for your lab because they are evaluating **Vibe Coding**, not whether an AI can generate an entirely new game.

6. First thing you should do on your computer

Open Command Prompt / PowerShell.

Clone your repository.

If you're starting from the faculty repository:

⟨/⟩ Bash

```
git clone https://github.com/SETAPESU26/24_treasure_hunt.git
```

Then:

```
</> Bash
```

```
cd 24_treasure_hunt
```

Check the files:

```
</> Bash
```

```
dir
```

You should see approximately:

```
</> Plain text
```

```
.gitignore  
README.md  
game.py
```

The public repository currently contains those files.  [GitHub](#)

7. Install Pygame

Run:

```
</> Bash
```

```
pip install pygame-ce
```

Then:

```
</> Bash
```

```
python game.py
```

The game should open.

Do not modify anything yet.

Play the original game.

Move around.

Find the key.

Find the chest.

Understand what happens.

8. Record your BEFORE video

This is required by the lab handout.

The handout says:

Record 10 seconds of video before making any changes.

 Lab_4_VibeCoding_Student_handout

So record around **10 seconds** of the original game.

For example:

 Plain text

0-3 sec → move player

3-6 sec → explore dungeon

6-10 sec → collect/move toward key

Don't worry about showing new features yet — they don't exist.

Save it as:

 Plain text

before.mp4

or

 Plain text

9. Then we start Vibe Coding

This is where I can help you the most.

We should **not** ask one giant prompt like:

"Complete my entire lab."

Instead, we follow the README:

</> Plain text

Prompt 1 → Task 1

Prompt 2 → Task 2

Prompt 3 → Task 3

Prompt 4 → Task 4

That gives you a much better **LLM chat history** for submission.

The README itself says:

"Write prompts to add all the features listed in the ReadMe file of the repo one by one."  GitHub

And it requires keeping the complete LLM chat history.  GitHub

10. Our first Vibe Coding prompt

Once you've cloned the repo and opened `game.py`, **don't change anything manually yet.**

Give your AI coding tool this prompt:

Lab 4 Task 1 Vibe Coding Prompt



I am completing Lab 4: VibeCoding for this existing Treasure Hunt Python/Pygame project.

First inspect the existing `game.py` and `README.md` carefully. Do not rewrite the entire project and do not change the existing architecture unnecessarily.

Implement ONLY Task 1 from the README: FLOOR TRAPS.

Requirements:

1. Add floor traps to the existing procedurally generated dungeon.
2. Traps must be visually distinguishable from normal floor tiles.
3. Place traps only on valid walkable floor tiles.
4. When the player steps on a trap, reset the player to the existing starting position/start room.
5. Show a clear status message indicating that the player triggered a trap and was reset.
6. Preserve all existing game functionality including movement, walls, key, chest, dungeon generation, and restart.
7. Do not add unnecessary external dependencies; continue using pygame/pygame-ce.
8. Keep the implementation modular and understandable.
9. Inspect the existing collision and player-state logic and integrate the trap behavior into it rather than creating a separate unrelated game system.

Before making changes, briefly explain which existing classes/functions/data structures you will modify and why.

Then implement Task 1 only.

After implementation, explain exactly how I can manually test the trap feature and what behavior I should observe.

11. Why this prompt is good for your lab

Because you're demonstrating the actual thing the lab is testing:

</> Plain text

Inspect



Understand



Prompt



AI modifies existing code



Test



Fix if needed

The README specifically says the goal is to use the LLM as a coding assistant while **you remain responsible for understanding and testing the result.**  [GitHub](#)

12. After Task 1 works

Then commit it.

For example:

```
</> Bash
```

```
git status
```

Then:

```
</> Bash
```

```
git add game.py
```

Then:

```
</> Bash
```

```
git commit -m "Add floor traps"
```

Then later:

```
</> Bash
```

```
git push
```

The important thing is that **each task gets its own commit**, because your handout explicitly requires separate commits for each task.  [Lab_4_VibeCoding_Student_handout](#)

Your history should ideally look like:

</> Plain text

```
Initial repository
  ↓
Add floor traps
  ↓
Add enemy guard
  ↓
Add mini-map
  ↓
Add inventory UI
```

That looks much better than one giant:

</> Plain text

```
"completed lab"
```

commit.

13. Then Task 2

After Task 1 is working and committed, we'll give the AI a **separate prompt** for:

</> Plain text

```
ENEMY GUARD
```

Then test:

</> Plain text

```
Guard moves A → B → A
Player touches guard
  ↓
Player resets
```

Then:

</> Bash

```
git add game.py
git commit -m "Add enemy guard"
```

14. Then Task 3

Separate prompt:

 Plain text

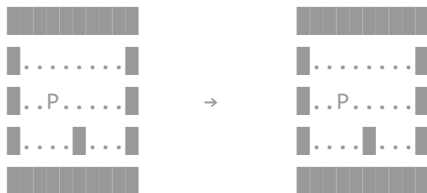
```
MINI-MAP
```

Test:

 Plain text

Actual dungeon

Mini-map



Then:

 Bash

```
git add game.py
git commit -m "Add dungeon mini-map"
```

15. Then Task 4

Separate prompt:

 Plain text

```
INVENTORY UI
```

Test:

`</>` Plain text

Before key:
[EMPTY]

After key:
[KEY]

Then:

`</>` Bash

```
git add game.py
git commit -m "Add inventory key UI"
```

16. Final testing

Before recording the final video, we do a complete checklist:

Requirement	Test
Original game	Game starts
Movement	WASD/arrows work
Trap	Player resets
Trap message	Message appears
Guard	Guard patrols
Guard collision	Player resets
Mini-map	Matches dungeon
Player marker	Moves with player
Key	Can collect key

Requirement	Test
Inventory	Key appears
Chest	Can open after key
Restart	R works
No crashes	Full playthrough

The README's required testing section specifically asks you to check these behaviors.  [GitHub](#)

17. Final BEFORE/AFTER videos

You need:

Video 1 — BEFORE

Around:

 Plain text

10 seconds

Original game, before modifications.

Video 2 — AFTER

Around:

 Plain text

10 seconds

Show as many new features as possible.

For example:

 Plain text

0-2 sec → player moves
2-4 sec → trap → reset/message
4-6 sec → guard moving
6-8 sec → mini-map
8-10 sec → inventory/key

You don't necessarily need to demonstrate every requirement perfectly in 10 seconds; the important thing is that the final game visibly shows the new functionality.

The repository's submission section also specifies a 10-second before video and a 10-second after video.

 [GitHub](#)

18. Chat history — VERY important

Your README specifically requires:

Complete LLM chat-history link included.  [GitHub](#)

And the handout requires the chat history as part of the deliverables.

 [Lab_4_VibeCoding_Student_handout](#)

So don't delete this conversation.

Your conversation should show something like:

 Plain text

Prompt 1

↓

AI inspected project

↓

Task 1 implementation

↓

Testing

↓

Prompt 2

↓

Task 2 implementation

↓

Testing

↓

Prompt 3

↓

Task 3

↓

Prompt 4



That is actually good evidence that you performed the lab through Vibe Coding.

19. What should ultimately be in your submission?

Your handout says the deliverables are:

</> Plain text

Lab-4/

|

|— Before video

|— After video

|— Chat history / LLM record

The handout describes the broader deliverables as videos, updated code, and exported chat history.

 Lab_4_VibeCoding_Student_handout

The repository README, however, says the final submission itself is the two 10-second videos plus the Chat/LLM page link.  GitHub

So follow the exact submission mechanism your faculty has told your class to use, but keep your updated code in your personal fork/repository as required by the handout.

20. One very important rule

Do NOT create a Pull Request to SETAPESU26 .

Your handout explicitly says:

Do not raise PR to the main repo (SETAPESU26).  Lab_4_VibeCoding_Student_handout

You should work on **your own fork/personal repository**.

So here's how we'll do your lab together

Don't try to do all four tasks at once.

Phase 1 — Setup

I'll help you:

Clone → run → understand → record BEFORE video

Phase 2 — Coding

We'll do:

Task 1 → test → commit

then

Task 2 → test → commit

then

Task 3 → test → commit

then

Task 4 → test → commit

Phase 3 — Submission

Full testing → AFTER video → GitHub → chat history → final submission

And I can guide you through **every single command and every prompt**, including what to click in GitHub/VS Code.

Your immediate next step

Open your terminal in the folder where you want the project and run:

 Bash

```
git clone https://github.com/SETAPESU26/24_treasure_hunt.git
cd 24_treasure_hunt
pip install pygame-ce
python game.py
```

Don't modify `game.py` yet. First get the original game running and record your 10-second BEFORE video. Then send me a screenshot of your terminal/game, and we'll start **Task 1: Floor Traps** properly.

can i do this in command prompt i cant do in vs code bcause some issue is there in my vs code thats why help me to complete this lab in command prompt

Absolutely. You can complete the entire Lab 4 using Command Prompt. You do NOT need VS Code.



In fact, for your situation, I recommend we keep it simple:

Command Prompt + Python + Git + ChatGPT = enough.

Your handout requires you to clone/run the Python game, make changes using Vibe Coding, test them, keep separate commits, and push to your own repo.  [Lab_4_VibeCoding_Student_handout](#)

How we'll do it

 Plain text

```
Command Prompt
↓
Clone repository
↓
Run original game
↓
Record BEFORE video
↓
Use ChatGPT prompts for Task 1
↓
Modify game.py from Command Prompt
↓
Run + test
↓
Git commit
↓
Task 2
↓
Git commit
↓
Task 3
```

One important thing

Since you don't want to use VS Code, we can have ChatGPT **give you the exact changes/code**, and then use Command Prompt to:

- open files

- check files
- run Python
- run Git commands
- test the game
- commit
- push

For editing `game.py`, you can use **Notepad** from Command Prompt if necessary:

```
</> cmd
```

```
notepad game.py
```

That's completely fine. You don't need VS Code.

STEP 1 — Check your computer

Open **Command Prompt** and run these one at a time:

```
</> cmd
```

```
python --version
```

Then:

```
</> cmd
```

```
pip --version
```

Then:

```
</> cmd
```

```
git --version
```

You should get something similar to:

```
</> Plain text
```

```
Python 3.x.x  
pip xx.x  
git version 2.x.x
```

Send me a screenshot of those three results.

Then I'll give you the **exact next commands for your computer.**

Don't clone or modify anything yet. We'll do the lab step-by-step so you don't get stuck.